

BDD SQL — Notes de cours (Séances 1 et 2)

BUT3 GEA — IUT de Saint-Denis

Ce document résume les notions vues en séances 1 et 2. Il sera complété au fil des séances suivantes. Il ne remplace pas les feuilles d'exercices : c'est une référence pour réviser.

Base de données relationnelle : l'idée de base

Une base de données relationnelle stocke des données dans des **tables**, un peu comme des feuilles Excel nommées, mais avec deux différences importantes :

- chaque **colonne** a un nom et un type fixé à l'avance (nombre entier, texte, etc.) ;
- une même table a toujours la même structure de colonnes, ligne après ligne.

On interroge une base de données avec le langage **SQL** (*Structured Query Language*), grâce à des requêtes qui permettent de retrouver rapidement des informations précises, même dans une table de plusieurs milliers de lignes — ce qui serait très laborieux à faire à la main dans Excel.

SQLiteStudio : les bases

- **Ouvrir l'éditeur SQL** : menu **Outils** → **Ouvrir l'éditeur SQL**, ou raccourci **Alt+E**.
- **Exécuter une requête** : cliquez sur le triangle bleu ►, ou appuyez sur **F9**. SQLiteStudio exécute la requête sur laquelle se trouve votre curseur — pas besoin d'un éditeur différent par requête.
- **Formater une requête** (l'afficher proprement sur plusieurs lignes) : raccourci **Ctrl+T**.
- **Ouvrir une base** : **Ctrl+O**.



Passer SQLiteStudio en français. Si le logiciel est en anglais : **Tools** → **Open Configuration dialog**, puis à gauche **Look & feel**, puis dans la liste déroulante **Language**, choisissez *French*. Redémarrez SQLiteStudio pour que le changement prenne effet.

Dans le panneau de gauche, chaque table a deux onglets qui nous intéressent : **Structure** (la liste des colonnes) et **Données** (le contenu, ligne par ligne).

SELECT et FROM

La commande de base pour interroger une table :

```
SELECT * FROM customers;
```

* signifie « toutes les colonnes ». Pour n'afficher que certaines colonnes, on les liste séparées par des virgules :

```
SELECT FirstName, LastName, Country FROM customers;
```

Filtrer avec WHERE

Pour n'afficher que certaines lignes, on ajoute une condition :

```
SELECT * FROM customers WHERE Country = "Brazil";
```

Opérateurs de comparaison :

Opérateur	Signification
=	égal
LIKE	égal, insensible à la casse
!=	différent
<	inférieur à
<=	inférieur ou égal
>	supérieur à
>=	supérieur ou égal

LIKE se comporte comme =, sauf qu'il ne fait pas la différence entre majuscules et minuscules :

```
SELECT * FROM customers WHERE Firstname = "jack";
-- ne trouve pas "Jack" (majuscule)

SELECT * FROM customers WHERE Firstname LIKE "jack";
-- trouve aussi "Jack"
```

Les jokers % et _

Utilisés avec LIKE, deux caractères spéciaux permettent de chercher un *motif* plutôt qu'une valeur exacte :

- % représente une suite de caractères quelconque, **y compris vide**.
- _ représente **exactement un** caractère.

```
SELECT * FROM customers WHERE Firstname LIKE "a%";
-- commence par "a"

SELECT * FROM customers WHERE Firstname LIKE "%a%";
-- contient un "a"

SELECT * FROM customers WHERE Firstname LIKE "_a%";
-- "a" en deuxième position

SELECT * FROM customers WHERE Email LIKE "%@gmail.com";
-- se termine par "@gmail.com"
```

Combiner des conditions : AND, OR, NOT, IN

```
SELECT * FROM invoices WHERE Total >= 10 AND Total <= 20;

SELECT *
FROM customers
WHERE Country = "France"
      OR Country = "Germany";
```

IN évite d'enchaîner plusieurs OR sur la même colonne :

```
SELECT *
FROM customers
WHERE Country IN ("France", "Germany", "UK");

-- équivalent à :
SELECT *
FROM customers
WHERE Country = "France"
      OR Country = "Germany"
      OR Country = "UK";
```

`NOT IN` fait l'inverse (exclut une liste de valeurs) :

```
SELECT *  
FROM customers  
WHERE Country NOT IN ("USA", "Canada");
```

DISTINCT : supprimer les répétitions

Une colonne peut contenir des valeurs répétées (par exemple, plusieurs factures pour la même ville). `DISTINCT` ne garde qu'une occurrence de chaque valeur :

```
SELECT DISTINCT BillingCountry FROM invoices;
```

ORDER BY : trier les résultats

```
SELECT DISTINCT BillingCity, BillingCountry  
FROM invoices  
ORDER BY BillingCountry ASC;
```

`ASC` (croissant) est l'ordre par défaut ; `DESC` inverse l'ordre (décroissant).

Colonnes calculées et AS

On peut afficher le résultat d'un calcul plutôt qu'une colonne brute, et lui donner un nom avec `AS` :

```
SELECT Milliseconds / 1000 AS Seconds FROM tracks;
```

Ceci ne modifie pas la table : le calcul est fait uniquement pour l'affichage du résultat.

Arrondir avec `ROUND` . La division ci-dessus donne un résultat entier (SQLite tronque, sans arrondir, quand les deux valeurs sont des entiers). Pour un arrondi contrôlé, avec un nombre choisi de décimales, on utilise `ROUND(valeur, décimales)` :

```
SELECT ROUND(Milliseconds / 1000.0, 2) AS Seconds FROM tracks;  
-- le ".0" force un calcul décimal (pas entier)  
-- ROUND(..., 2) arrondit à 2 chiffres après la virgule
```



Division entière vs décimale. En SQLite, `7 /`

`2` vaut `3` (division entière, le reste est perdu), alors que `7.0 / 2` ou `7 / 2.0` vaut `3.5` . Dès qu'un des deux nombres a une virgule (même `.0`), le calcul devient décimal. C'est une source d'erreur fréquente : pensez-y si un résultat calculé semble « arrondi à l'entier » sans que vous l'ayez demandé.

Clé primaire, clé étrangère

Une **clé primaire** (*primary key*) identifie chaque ligne d'une table de façon unique (par exemple, `CustomerId` dans la table `customers`).

Une **clé étrangère** (*foreign key*) est une colonne d'une table qui référence la clé primaire d'une autre table, pour représenter un lien entre les deux. Par exemple, dans `invoices` , la colonne `CustomerId` référence `customers.CustomerId` : chaque facture est associée à un client précis.

C'est la même idée qu'une relation entre deux feuilles dans Access, ou qu'un `VLOOKUP` / `RECHERCHEV` entre deux feuilles Excel via un identifiant commun.

Dans SQLiteStudio, l'onglet **Structure** d'une table indique sa clé primaire (une petite icône de clé, dans la colonne correspondante), et l'onglet **Contraintes** liste ses clés étrangères.

INNER JOIN : combiner deux tables

Une jointure permet de combiner les lignes de deux tables lorsqu'une colonne de l'une correspond à une colonne de l'autre — typiquement, une clé étrangère et la clé primaire qu'elle référence.

```
SELECT customers.FirstName, customers.LastName, invoices.Total
FROM invoices
JOIN customers ON invoices.CustomerId = customers.CustomerId;
```

`JOIN` (ou `INNER JOIN`, équivalent) ne garde que les lignes qui trouvent une correspondance dans les deux tables. On peut ajouter un `WHERE` après la jointure, exactement comme d'habitude :

```
SELECT customers.FirstName, customers.LastName, invoices.Total
FROM invoices
JOIN customers ON invoices.CustomerId = customers.CustomerId
WHERE invoices.Total > 15;
```



Pourquoi préfixer `customers.FirstName` ? Dès qu'une requête combine plusieurs tables, deux colonnes de tables différentes peuvent porter le même nom. Préfixer par le nom de la table (`table.colonne`) lève toute ambiguïté, et c'est une bonne habitude à prendre dès qu'une jointure est en jeu.

Anatomie d'une requête complète

Voici, annotée, la requête de l'exemple ci-dessus — pour visualiser d'un coup le rôle de chaque partie :

<code>SELECT</code> customers.FirstName, customers.LastName, invoices.Total	← quelles colonnes afficher
<code>FROM</code> invoices	← table de départ
<code>JOIN</code> customers <code>ON</code> invoices.CustomerId = customers.CustomerId	← table à combiner, condition de jointure
<code>WHERE</code> invoices.Total > 15	← quelles lignes garder
<code>ORDER BY</code> invoices.Total DESC;	← dans quel ordre trier

Anatomie d'une requête : chaque clause répond à une question différente.

L'ordre d'écriture (`SELECT ... FROM ... JOIN ... WHERE ... ORDER BY`) est fixe : ces clauses doivent apparaître dans cet ordre-là. Ce n'est en revanche pas l'ordre dans lequel SQLite les exécute — mais ce point de détail n'a pas besoin de vous préoccuper pour l'instant.